SKALA

sLLM 구현 및 Fine Tuning

2026.08

임성열 교수 Ph.D.

SK 주식회사 AX



1. sLLM 구현 및 Fine Tuning
2. sLLM 서비스 설계 및 PEFT 응용

1. sLLM 구현 및 Fine Tuning

- LLM 파이프라인 개요
- LLM vs sLLM 비교
- Fine Tuning 전략
- PEFT 기초 실습

SK 주식회사 AX

교육 개요

내부 파이프라인의 필요성부터 CLM Fine-Tuning, PEFT 실습 진행까지 완성

- AI 개발을 위한 인프라 이해
- 내부 LLM 파이프라인 구현 및 상용 배포를 위한 Tuning 실무 연습

교육 기간



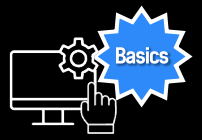
2일 · 16시간 (Day1+Day2)

교육 방식



이론 50% · 실습 50%

Day1 목표



sLLM 이해 · PEFT 기초 실습

Day 1 : sLLM 구현 및 Fine Tuning
이론 4h + 실습 4h - 내부 파이프라인의 필요성, CLM Fine-Tuning, PEFT 기초 실습

Day 2 : sLLM 서비스 설계 및 PEFT 응용
이론 4h + 실습 4h - 서비스 파이프라인 구현, sLLM 서빙 배포

Day1 핵심 산출물

LLM/sLLM 구조 차이 이해 + 간단한 PEFT(LoRA 등) 실습 및 코드 리뷰 완료

Day 1 목차 한눈에 보기

목차	주요 학습내용 (이론 + 실전 보강)	시간	방식
LLM 파이프라인 개요	서빙 파이프라인 이해 + 외부/내부 파이프라인 구분 근거 + 내부 파이프라인 목적	1h	이론
MLM vs CLM 이해	구조 비교 + Fine-Tuning이 CLM 중심인 이유 + 산업 동향 + 상용 CLM 기법	1h	이론
LLM vs. sLLM 비교	비교 + 버티컬 sLLM 요건 + Fine-Tuning이 특성을 완성하는 다리	1h	이론
sLLM Use Case 실습	상용 LLM 제약 상황 + 활용 사례 3선 + 사내 문서 Q&A 구현 시나리오	1h	이론/실습
Fine Tuning 전략	시나리오별 PEFT 기법 매핑 + LoRA/Prompt Tuning 코드 + 성능 비교표	1h	이론/실습
PEFT 기초 실습	실습 시나리오 정의 + 모델·LoRA 설정 코드 + 학습·추론 코드 + 리뷰 체크포인트	3h	종합실습

LLM 파이프라인 개요

LLM 파이프라인 개요 · 1/4

- AI 서비스를 위한 LLM 서빙 파이프라인(Serving Pipeline)의 전체 구조 이해
- 요청 수신 → 전처리 → 모델 추론 → 후처리 → 응답 반환의 단계별 흐름
- 파이프라인을 '외부'와 '내부'로 구분하는 이유는 다음 슬라이드에서 상세히 다룹니다.
- 다음 슬라이드에서 외부/내부 파이프라인 구분 근거와 내부 파이프라인 구성 목적을 다룹니다.

외부 파이프라인 vs 내부 파이프라인 - 왜 구분하는가?

LLM 파이프라인 개요 · 2/4 · 개념

외부 파이프라인 - 상용 LLM 활용

Client → API Gateway → 상용 LLM(GPT/Claude 등) → 응답

- 강력한 범용 성능, 빠른 도입, 별도 인프라 구축 불필요
- 요청 데이터가 사외 서버(벤더 인프라)로 전송됨
- 범용 지식에는 강하지만 사내 고유 도메인 지식은 약함

내부 파이프라인 - sLLM 활용

Client → 내부 Gateway → sLLM(사내 인프라) ↔ 내부 DBMS/문서 → 응답

- 요청·응답 데이터가 사내망을 벗어나지 않음
- 내부 DBMS·문서 저장소와 직접 연결이 가능
- 인프라 구축·운영 부담은 있으나 도메인 특화 가능

상용 LLM 내부망 구축의 한계

LLM 파이프라인 개요 · 3/4 · 개념

- **VPC·전용 인스턴스로 상용 LLM을 사내망에 배치해도, 모델의 학습 데이터 활용 정책은 벤더 정책에 의존**
- **계약상 "학습에 사용하지 않음"을 명시해도, 이를 기술적으로 완전히 감사·검증하기는 현실적으로 어려움**
- **규제 산업(금융/의료/공공)은 이 불확실성 자체를 리스크로 간주해 완전 내부 격리를 요구하는 경우가 많음**

비교 항목	상용 LLM (사내망 배치)	sLLM (완전 내부)
데이터 통제권	벤더 정책에 의존	완전히 사내 보유
기술적 감사 가능성	제한적 (블랙박스)	인프라·로그 전체 감사 가능
도메인 커스터마이징	프롬프트 수준 제한적 대응	Fine-Tuning으로 심층 반영
규제 대응 리스크	계약서 의존 (기술적 보증 아님)	구조적으로 리스크 최소화

내부 파이프라인 구성의 두 가지 목적

LLM 파이프라인 개요 · 4/4 · 개념

목적 1 · 데이터 주권 확보

외부 전송이 원천적으로 불가능한 구조를 만들어, 계약이 아닌 아키텍처로 데이터 보안을 보증

목적 2 · 도메인 버티컬 최적화

특정 도메인 지식(법률/의료/사내 프로세스)을 연결해, 범용 LLM보다 좁지만 깊은 목적 성능을 확보

범용 LLM은 넓지만 얕은 지식을 갖는 반면, 버티컬 sLLM은 좁지만 깊은 지식과 낮은 지연시간 비용을 함께 얻을 수 있습니다. 이 두 목적이 이번 과정 전체 실습의 배경이 됩니다.

MLM vs CLM 이해

MLM vs CLM 이해 · 1/5

- Masked Language Model(MLM) - 문맥 양방향 참조, 빈칸 채우기 방식 학습 (예: BERT 계열)
- Causal Language Model(CLM) - 이전 토큰만 참조하는 단방향 순차 생성 학습 (예: GPT 계열)
- 왜 Fine-Tuning 영역은 MLM보다 CLM을 주로 다루는지, 다음 슬라이드에서 상세히 다룹니다.
- 다음 슬라이드에서 구조 비교, CLM 중심 이유, 산업 동향, 상용 기법을 다룹니다.

MLM vs CLM 구조 상세 비교

MLM vs CLM 이해 · 2/5 · 개념

구분	MLM (Masked LM)	CLM (Causal LM)
학습 목표	문장 내 마스킹된 토큰 예측	다음 토큰을 순차적으로 예측
문맥 참조 방향	양방향 (앞뒤 문맥 모두 참조)	단방향 (이전 토큰만 참조)
대표 구조	Encoder (또는 Encoder-only)	Decoder (Decoder-only)
대표 모델	BERT, RoBERTa	GPT, Llama, Mistral
강점 분야	문장 이해, 분류, 개체명 인식	자유 생성, 대화, 코드 생성

왜 Fine Tuning은 CLM 중심으로 이루어지는가?

MLM vs CLM 이해 · 3/5 · 개념

1. **생성형 태스크의 다운스트림 다양성**

   요약·대화·코드생성 등 서로 다른 태스크를 하나의 디코더 구조로 자연스럽게 대응 가능

2. **Instruction-following 패러다임과의 정합성**

   "프롬프트 → 응답" 형태의 학습이 CLM의 다음 토큰 예측 구조와 직접적으로 일치

3. **MLM 구조의 생성 한계**

   양방향 마스킹 구조는 자유 생성에 부적합해 분류·추출 태스크에 국한되는 경향

4. **산업 표준화**

   최근 sLLM(Llama, Qwen, Gemma, Exaone 등)이 대부분 Decoder-only(CLM) 구조를 채택

   * 상용 LLM인 ChatGPT, Gemini 들도 Decoder-only 계열임 (Claude는 내부 구조 비공개 : Decoder-only 특성 보유)

연구·산업 발전 동향 - BERT 시대에서 GPT 시대로

MLM vs CLM 이해 · 4/5 · 동향

2018

BERT 등장

MLM 기반 문장 이해 태스크가 주류를 이룸

2019-20

GPT-2 / GPT-3

대규모 CLM의 생성 능력이 부각되기 시작

2022

InstructGPT / ChatGPT

Instruction Tuning·RLHF로 대화형 CLM 활용 확산

2023

오픈소스 sLLM 확산

Llama, Qwen 등 경량 CLM이 사내 파이프라인 후보로 부상

생성형 서비스 수요 증가, RLHF·Instruction Tuning 기법의 정립, Decoder-only 구조가 대규모 학습에 유리하다는 다수 연구 결과가 맞물리며 산업 전반이 CLM 중심으로 수렴했습니다.

상용 환경의 CLM Fine Tuning 기법 개요

MLM vs CLM 이해 · 5/5 · 개념

기법	핵심 설명	학습 대상
Instruction Tuning	지시-응답 쌍으로 모델이 지시를 따르도록 학습	전체/부분 파라미터
RLHF / DPO	사람 선호도를 반영해 응답 품질을 정렬	보상모델 + 정책 최적화
PEFT (LoRA 등)	적은 파라미터만 학습해 효율적으로 미세조정	경량 어댑터 추가

사내 sLLM 파이프라인에서는 대규모 RLHF보다 Instruction Tuning + PEFT(LoRA) 조합이 비용·속도 면에서 실무적으로 가장 많이 사용됩니다. 이 조합을 이후 Fine Tuning 전략 챕터에서 상세히 다룹니다.

LLM vs. sLLM 비교



LLM vs SLLM 비교 · 1/4

- LLM 공통 기능 이해 - 토크나이저, 임베딩 알고리즘, 전처리/후처리 과정
- sLLM 핵심 기능 도출 - 경량화 아키텍처, 양자화(Quantization), On-Device 추론
- sLLM이 버티컬 도메인/내부 업무에 쓰이기 위해 갖춰야 할 특성은 다음 슬라이드에서 다룹니다
- 다음 슬라이드에서 sLLM 요건, 다차원 비교표, Fine Tuning의 역할을 다룹니다

버티컬 도메인 sLLM이 갖춰야 할 특성

LLM vs SLLM 비교 · 2/4 · 개념

도메인 지식 반영도

전문 용어·사내 프로세스·업무 맥락을 정확히 이해

추론 효율성

On-Device 또는 저비용 인프라에서도 충분히 빠른 응답

데이터 프라이버시

민감 정보가 외부로 유출되지 않는 구조적 보장

최신성 유지 용이성

낮은 재학습 비용으로 정책·지식 변경에 신속히 대응

안전한 출력 제어

사내 정책·규정에 맞는 응답 범위 준수

LLM vs sLLM 다차원 비교표

LLM vs SLLM 비교 · 3/4 · 비교표

비교 항목	LLM	sLLM
파라미터 규모	수백억~수조 단위	수억~수십억 단위
추론 비용/지연시간	높음 (클라우드 GPU 의존)	낮음 (On-Device 가능)
커스터마이징 난이도	Fine-Tuning 비용이 매우 큼	PEFT로 저비용 커스터마이징
도메인 특화 가능성	범용 지식에 강점	버티컬 지식 주입에 강점
데이터 프라이버시	외부 인프라 의존 시 제약	완전 내부 운용 가능

Fine Tuning - sLLM 특성을 완성하는 핵심 다리

LLM vs SLLM 비교 · 4/4 · 개념

기본 sLLM
(경량 · 범용지식 얕음)

›

Fine-Tuning
(도메인 데이터 주입)

›

도메인 특화
sLLM (요구 특성 충족)

- Fine-Tuning 없는 sLLM은 단지 '작은 모델'일 뿐, 실무에 쓸 수 있는 수준의 도메인 정확도를 보장하지 않음
- Fine-Tuning을 통해 도메인 지식 주입, 응답 스타일 정렬, 사내 안전 정책 반영이 이루어져야 비로소 실무 적용이 가능해짐
- 즉, 앞서 정리한 sLLM의 요건(도메인 지식/프라이버시/최신성/안전성)은 Fine-Tuning을 통해 '완성' 되는 특성입니다.

sLLM Use Case 실습



sLLM USE CASE 실습 · 1/4

- sLLM의 특징(경량성, 온디바이스, 데이터 보안성)을 활용한 Use Case 이해
- 상용 LLM을 쓰기 어려운 환경/사유는 다음 슬라이드에서 유형별로 정리합니다.
- 실제 구현 시나리오 중심으로 사내 문서 기반 sLLM Q&A 챗봇 사례를 다룹니다.
- 다음 슬라이드에서 제약 상황 유형, 활용 사례 3선, 구현 시나리오를 다룹니다.

상용 LLM 사용이 어려운 상황 유형

sLLM USE CASE 실습 · 2/4 · 실전 적용

제약 유형	설명	대표 산업/상황
망분리 · 보안 규제 환경	외부 네트워크 연결 자체가 금지	금융, 국방, 공공기관
오프라인 · 실시간 처리 필요	네트워크 단절 환경에서도 동작해야 함	제조 현장, 선박·플랜트
초저지연 요구	API 왕복 지연을 감당할 수 없는 서비스	실시간 상담·제어 시스템
반복 호출 비용 부담	대량 호출 시 API 과금이 누적	고빈도 내부 문서 처리
도메인 특수 용어/PII 포함	외부 반출이 금지된 민감 데이터	의료, 법률, 고객 상담 로그

시나리오별 sLLM 활용 사례

sLLM USE CASE 실습 · 3/4 · 실전 적용

HR 내부 문서 자산

내부HR 규정 등 외부 전송이 금지된 문서를 sLLM이 사내에서 직접 요약

제조 설비 매뉴얼 Q&A

공장 네트워크 단절 환경에서 On-Device sLLM이 설비 매뉴얼 기반 질의응답 제공

고객 상담 로그 분석

개인정보(PII)가 포함된 상담 기록을 외부 반출 없이 sLLM으로 분류·요약

구현 시나리오 상세 - 사내 문서 기반 sLLM Q&A 챗봇

sLLM USE CASE 실습 · 4/4 · 실행 시나리오



1. **문서 수집 및 청크 분할**
   사내 규정·매뉴얼 문서를 의미 단위(청크)로 분할

2. **임베딩 생성 및 벡터DB 저장**
   임베딩 모델로 각 청크를 벡터화하여 벡터DB에 색인

3. **질의 시 관련 문서 검색(RAG)**
   사용자 질의와 유사한 청크를 벡터DB에서 검색

4. **sLLM 추론 및 답변 생성**
   검색된 문서를 컨텍스트로 제공해 sLLM이 최종 답변 생성

Fine Tuning 전략

FINE TUNING 전략 · 1/5

- Fine Tuning 전략 이해 - Full Fine-Tuning과 PEFT의 차이
- PEFT 주요 기법 - LoRA, Adapter, Prefix-Tuning, Prompt-Tuning
- 시나리오별 효과적인 기법과 예제 코드는 다음 슬라이드에서 다룹니다.
- 다음 슬라이드에서 기법 매핑, LoRA/Prompt Tuning 코드, 성능 비교표를 다룹니다.

시나리오별 적합 PEFT 기법 매핑

FINE TUNING 전략 · 2/5 · 실전 적용

시나리오	목적	적합 기법
도메인 지식 주입 (HR 문서 요약)	전문 용어·맥락을 모델에 반영	**LoRA / Adapter**
극소 자원 환경 (On-Device)	최소 파라미터로 경량 적응	**Prompt Tuning / Prefix Tuning**
추론 능력 강화 (복잡 QA)	지시 이해 및 응답 품질 향상	**Instruction Tuning + LoRA 결합**

다음 두 슬라이드에서 LoRA와 Prompt Tuning의 실제 적용 코드를 각각 살펴봅니다. 두 기법 모두 PEFT 라이브러리 기준으로 제시했습니다.

LoRA 적용 예제 코드 - 도메인 지식 주입

FINE TUNING 전략 · 3/5 · 예제 코드 (Skeleton)

기존 모델 위에 LoRA Adapter를 붙인 후, 학습 시에 도메인 주식을 주입

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model, TaskType

model = AutoModelForCausalLM.from_pretrained("internal-sllm-base")
tokenizer = AutoTokenizer.from_pretrained("internal-sllm-base")

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,                  # rank: 낮을수록 경량, 높을수록 표현력 증가
    lora_alpha=16,         # 스케일링 계수
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],  # 어텐션 투영층에 적용
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# 예: trainable params: 4.2M || all params: 1.3B || trainable%: 0.32%
```

```
SFTTrainer(
    model=model,
    train_dataset=domain_dataset, …
)
```

☑ 도메인 데이터

↓

Forward

↓

Loss 계산

↓

Backpropagation

↓

LoRA Weight 업데이트

Prompt Tuning 예제 코드 - 극소 자원 환경

FINE TUNING 전략 · 4/5 · 예제 코드 (Skeleton)

기본 모델 앞에 **학습 가능한 virtual prompt embedding** 을 붙여, 도메인 지식이 "모델 내부 가중치 전체"에 주입되는 게 아니라, **입력 앞에 붙는 작은 Prompt Embedding 벡터**에 학습

```
from peft import PromptTuningConfig, PromptTuningInit, get_peft_model

prompt_config = PromptTuningConfig(
    task_type="CAUSAL_LM",
    prompt_tuning_init=PromptTuningInit.TEXT,
    num_virtual_tokens=20,        # 학습되는 가상 토큰 개수 (매우 적음)
    prompt_tuning_init_text="다음 사내 문서를 바탕으로 정확하게 답변하라:",
    tokenizer_name_or_path="internal-sllm-base",
)

model = get_peft_model(base_model, prompt_config)
model.print_trainable_parameters()
# 예: trainable params: 20K || all params: 1.3B || trainable%: 0.0015%

# On-Device 환경처럼 저장·연산 자원이 극도로 제한적일 때 적합
```

☑ 도메인 데이터

↓

Virtual Prompt + 사용자 입력 생성

↓

Forward

↓

Loss 계산

↓

Backpropagation

↓

Virtual Prompt Embedding 업데이트

0. 프로젝트 구조 이해

- 전체 디렉토리 구조 한눈에 보기
- 코드를 읽는 순서 (데이터 → 학습 → 평가 → 서비스)
- Instruction SFT vs Context-based QA SFT
- PEFT 기초 실습으로 이동

SK 주식회사 AX

개선 전 vs 개선 후 성능·정확도 비교 (예시)

FINE TUNING 전략 · 5/5 · 비교표

시나리오 (적용 기법)	Fine-Tuning 전	Fine-Tuning 후	개선폭
HR 규정 요약 (Instruction SFT + Context-based QA SFT)	58%	**84%**	**+26%p**
설비 매뉴얼 Q&A (Prompt Tuning)	47%	**71%**	**+24%p**
상담 로그 분류 (Instruction SFT)	62%	**89%**	**+27%p**

⚠ 위 수치는 교육 목적의 예시 값입니다. 실제 프로젝트에서는 검증 데이터셋 기준으로 반드시 자체 측정한 정확도를 사용해야 합니다.

PEFT 기초 실습

PEFT 기초 실습

STEP 1 · 1시간

개념 실습

- sLLM의 기본 동작 원리를 직접 실행하며 확인
- 간단한 PEFT(LoRA) 적용 전/후 모델 파라미터 변화 비교
- 학습 데이터 소량으로 미세조정 결과 관찰

STEP 2 · 2시간

코드 리뷰

- 제공된 PEFT(LoRA) 구현 코드를 팀별로 리뷰
- LoRA 설정 파라미터(rank, alpha 등)의 의미와 영향 분석
- 학습 결과(loss, 성능 지표)를 함께 확인하고 토론

실습 시나리오 정의 -사내 FAQ 챗봇 LoRA 튜닝

PEFT 기초 실습 · 실습 시나리오

목표	데이터셋	기대 결과
사내 인사/총무 FAQ에 정확히 답하는 sLLM 챗봇 만들기	사내 FAQ Q&A 쌍 약 200개 (예: 연차, 경비, 재택근무 규정)	Fine-Tuning 전보다 도메인 질문 정답률이 눈에 띄게 향상

실습 진행 순서

1. **FAQ 데이터 준비**
   질문-답변 쌍을 Instruction 형식(질문/답변)으로 정리
2. **모델·LoRA 설정**
   베이스 sLLM 로드 후 LoRA Config 적용 (다음 슬라이드)
3. **학습 실행**
   소량의 데이터로 빠르게 학습 후 결과 확인 (다음 슬라이드)
4. **결과 검증**
   학습 전/후 동일 질문에 대한 응답 품질 비교

실습 시나리오 정의 - 사내 FAQ 챗봇 LoRA 튜닝

1. **모델 성능 향상**
   → Qwen2.5 + LoRA FAQ Fine Tuning (Day1)
2. **Context & Reasoning**
   → Qwen2.5 + FAISS RAG FAQ 검색 답변 (Day1)
   → 맥락 압축, 요약 후 답변, 긴 입력 정리, Self-refine (Day2)
3. **Inference Optimization**
   → Qwen2.5 4bit Quantization 추론 테스트 (Day1)

구분	sLLM Fine Tuning 유형	학습 여부	좋아지는 것
1	모델의 정확도, 응답 품질, 처리 속도 등 전반적인 역량을 개선	O	FAQ 도메인 적응
2	모델이 주어진 문맥(맥락)을 정확히 이해하고, 이를 바탕으로 논리적인 추론을 수행	X	외부 문서 기반 답변 or LLM 특성 활성화
3	동일하거나 더 나은 추론 성능을 더 적은 연산량·시간·비용으로 달성하도록 최적화	X	메모리 절감, 실행 비용 감소

sllm-main 전체 프로젝트 구조 (데모용 포함 전체)

0. 프로젝트 구조 이해 · 2/5 · 전체 디렉토리 구조

```
sllm-main/
├── requirements.txt
├── readme.md
├── dataset/
│   ├── source/          원천 문서                    [데모 · 전처리용, 수업 범위 아님]
│   ├── training/        학습·검증 데이터             [실습 제공]
│   └── evaluation/      평가 데이터                  [실습 제공]
├── scripts/
│   ├── build_sft_dataset.py          데이터 분리     [데모 · 실습 X]
│   ├── build_training_dataset.py     데이터 생성     [데모 · 실습 X]
│   ├── train_lora.py                                 [실습 제공]
│   └── evaluate_model.py                             [실습 제공]
├── models/hr-qwen-lora/   학습 결과                  [실습 제공 · 빈 폴더]
├── outputs/                                          [실습 제공 · 빈 폴더]
├── vector_db/, run.ipynb                             [데모 · 수업 범위 아님]
└── sllm_service/, frontend/, app.py                  [데모 · 수업 범위 아님]
```

이번 실습은 원천 문서의 수집·전처리가 아니라, 준비된 학습 데이터를 이용해 SFT를 수행하고 Base Model 대비 Fine-tuned Model의 성능 변화를 확인하는 것이 목적입니다.
따라서 원천 데이터와 데이터 생성 과정은 실습 범위에서 제외하고, SFT에 바로 사용할 수 있도록 전처리된 Train/Validation/Evaluation 데이터를 제공합니다.

실습 배포본 구조 (sllm-student/)

0. 프로젝트 구조 이해 · 3/5 · 학생 배포본

```
sllm-student/
├── requirements.txt
├── dataset/
│   ├── training/
│   │   ├── hr_sft_train.jsonl
│   │   └── hr_sft_validation.jsonl
│   └── evaluation/
│       └── hr_eval.jsonl
├── scripts/
│   ├── train_lora.py
│   └── evaluate_model.py
├── models/hr-qwen-lora/     (처음에는 빈 폴더)
└── outputs/                 (처음에는 빈 폴더)
```

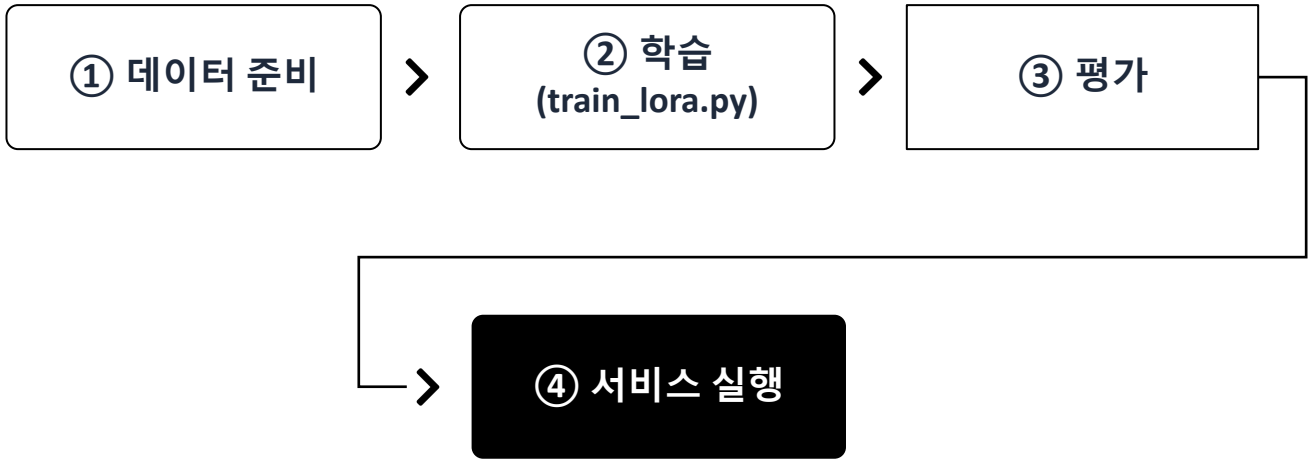
RAG 및 서빙 은 실습 제외 (수업 범위 아님)

sllm_service/, frontend/, app.py

→ RAG·서빙 코드는 별도 과목 영역이라 제공하지 않으며,
필요 시, 구축된 URL로 데모를 접속하여 설명할 수 있습니다.

코드를 읽는 흐름

scripts/build_sft_dataset.py · build_training_dataset.py로 데이터를 만들고, train_lora.py로 학습한 뒤 evaluate_model.py로 검증하고, sllm_service로 서빙합니다.



Instruction SFT vs Context-based QA SFT - 두 갈래

0. 프로젝트 구조 이해 · 4/4 · 두 가지 SFT 방식

구분	Instruction SFT (build_sft_dataset.py)	Context-based QA SFT (build_training_dataset.py)
입력 데이터	이미 만들어진 messages(Q&A) 세트: hr_sft_all.jsonl	원천 문서: dataset/source/hr_manual_v1.txt
데이터 생성 방식	형식 검증 후 train/validation/evaluation 분리	문서를 Section으로 분리해 질문·정답 자동 생성
정답 구성	사람이 미리 작성한 assistant 답변	결론 + 적용 규정(context 원문)을 포함한 구조화 답변
학습 목적	정제된 지시-응답 스타일 학습	문서 근거 기반 응답 + Hallucination 방지 학습
한계	원본 Q&A 품질에 결과가 좌우됨	문서 구조·품질에 따라 질문 다양성 제한
관계	두 데이터셋 모두 동일한 train_lora.py로 학습	하나의 hr-qwen-lora 모델로 통합됨

실습 시나리오 정의 - HR 규정 안내 챗봇 LoRA 튜닝

PEFT 기초 실습 · 2/5 · 실습 시나리오

목표

Human AI Corporation 인사 규정에 정확히 답하는 sLLM 챗봇 만들기

데이터셋

HR 매뉴얼(hr_manual_v1.txt) 기반 자동 생성 SFT 데이터 (연차·재택근무·보안교육 등)

기대 결과

Base 모델 대비 규정 키워드 반영률 상승 및 Hallucination(허위 답변) 감소

실습 진행 순서

1. **Context-based QA SFT 데이터 생성**
   hr_manual_v1.txt를 규정 Section별로 나눠 질문·정답 자동 생성
2. **Instruction SFT 데이터 분리**
   검증된 Q&A를 train/validation/evaluation으로 분리 (다음 슬라이드)
3. **LoRA/QLoRA 학습**
   환경별(CUDA/MPS/CPU) 자동 분기로 hr-qwen-lora 학습 (다음 슬라이드)
4. **결과 평가**
   Base vs Fine-tuned 모델의 키워드·F1·Hallucination 점수 비교

실습 시나리오 정의 - HR 규정 안내 챗봇 LoRA 튜닝

1. **모델 성능 향상**
   → Qwen2.5 + LoRA/QLoRA HR 규정 Fine-Tuning (Day1)

2. **Context & Reasoning**
   → Qwen2.5 + BGE-M3 RAG 문서 검색 답변 (Day1)
   → 맥락 압축, 요약 후 답변, 긴 입력 정리, Self-refine (Day2)

3. **Inference Optimization**
   → Qwen2.5 4bit Quantization 추론 테스트 (Day1)

구분	sLLM Fine Tuning 유형	학습 여부	좋아지는 것
1	모델의 정확도, 응답 품질, 처리 속도 등 전반적인 역량을 개선	O	HR 규정 도메인 적응
2	모델이 주어진 문맥(맥락)을 정확히 이해하고, 이를 바탕으로 논리적인 추론을 수행	X	외부 문서 기반 답변 or LLM 특성 활성화
3	동일하거나 더 나은 추론 성능을 더 적은 연산량·시간·비용으로 달성하도록 최적화	X	메모리 절감, 실행 비용 감소

실습 코드 - Instruction SFT 데이터 검증·분리

PEFT 기초 실습 · 3/5 · 실습 코드 (Instruction SFT · build_sft_dataset.py)

```
# scripts/build_sft_dataset.py
# 입력: dataset/training/hr_sft_all.jsonl (검수 완료 Q&A)
import json, random

def validate_record(record, line_number):
    messages = record.get("messages")
    if not isinstance(messages, list) or len(messages) < 2:
        raise ValueError(f"{line_number}번째 줄: messages 필요")
    roles = [m.get("role") for m in messages]
    if "user" not in roles or "assistant" not in roles:
        raise ValueError(f"{line_number}번째 줄: user/assistant 없음")

def main():
    records = read_jsonl(INPUT_PATH)   # 형식 검증 포함
    random.seed(RANDOM_SEED)
    random.shuffle(records)

    total = len(records)
    eval_n = max(1, round(total * 0.1))
    val_n  = max(1, round(total * 0.1))

    write_jsonl(TRAIN_PATH,      records[eval_n + val_n:])
    write_jsonl(VALIDATION_PATH, records[eval_n:eval_n + val_n])
    write_jsonl(EVALUATION_PATH, records[:eval_n])
```

실습 코드 - Context-based QA SFT 데이터 생성 #1

PEFT 기초 실습 · 3/5 · 실습 코드 (Context-based QA SFT #1 · 문서 분리)

```
# scripts/build_training_dataset.py
# 입력: dataset/source/hr_manual_v1.txt
from dataclasses import dataclass

@dataclass(frozen=True)
class ManualSection:
    section_id: str
    title: str
    content: str

# "Chapter 3","제3장","3.1 연차" 패턴을 규정 제목으로 인식
def is_heading(line: str) -> bool:
    return any(p.search(line.strip()) for p in HEADING_PATTERNS)

def split_manual_sections(text: str) -> list[ManualSection]:
    sections, title, lines = [], "", []
    def flush():
        content = normalize_content(lines)
        if is_valid_section(title, content):
            sid = f"section-{len(sections)+1:03d}"
            sections.append(ManualSection(sid, title.strip(), content))
    for line in text.splitlines():
        if is_heading(line):
            flush(); title = line.strip()
        else:
```

실습 코드 - Context-based QA SFT 데이터 생성 #2

PEFT 기초 실습 · 3/5 · 실습 코드 (Context-based QA SFT #2 · 질문·정답 생성)

```
# 같은 규정을 6가지 질문 표현으로 반복 학습
def create_train_questions(section):
    title = clean_section_title(section.title)
    return [
        f"{title}에 대한 규정을 설명해 주세요.",
        f"직원이 {title}과 관련해 반드시 알아야 할 사항은?",
        f"{title}을 실제 업무에 적용할 때 조건과 예외는?",
        f"{title} 규정의 핵심 기준과 예외를 정리해 주세요.",
    ]

# 정답에 원문 규정(context)을 그대로 포함하는 것이 핵심
def build_structured_answer(section):
    conclusion = first_sentence(section.content)
    return f"""
결론: {conclusion}

적용 규정: {section.content}

위 규정만으로 판단하기 어려운 개별 상황은 추측하지 말고
인사팀 또는 담당 부서에 확인해야 합니다.
""".strip()
```

실습 코드 - Context-based QA SFT 데이터 생성 #3

PEFT 기초 실습 · 3/5 · 실습 코드 (Context-based QA SFT #3 · 평가 키워드·안전장치)

```
# 평가용 must_include 키워드: 회사 코드 > 숫자·기한 > 정책 용어 순
def extract_must_include_keywords(section, max_keywords):
    keywords = []
    keywords += extract_company_codes(section.content)  # AURORA, ORBIT ..
    keywords += extract_numeric_rules(section.content)  # "3영업일" 등
    keywords += [t for t in COMPANY_POLICY_TERMS if t in section.content]
    return list(dict.fromkeys(keywords))[:max_keywords]

# 규정에 없는 질문에는 "모른다"고 답하도록 안전 응답 학습
def unknown_policy_answer() -> str:
    return (
        "현재 제공된 규정에서는 해당 내용을 확인할 수 없습니다. "
        "임의로 추측하거나 단정하지 말고 인사팀 또는 담당 부서에 "
        "확인해야 합니다."
    )

# "개인 여행 경비를 전액 지원하나요?" 같은 질문 + 위 안전 답변을
# train / validation / evaluation 각 세트에 함께 넣어
# Hallucination(허위 확답)을 억제하도록 학습합니다.
```

실습 코드 - LoRA/QLoRA 학습 환경 자동 분기

PEFT 기초 실습 · 3/5 · 실습 코드 (train_lora.py #1 · 환경 감지)

```
# scripts/train_lora.py
def detect_device_type() -> str:
    if torch.cuda.is_available():
        return "cuda"
    if torch.backends.mps.is_available():
        return "mps"
    return "cpu"

def create_training_profile(device_type):
    if device_type == "cuda":
        use_qlora = parse_bool_env("USE_QLORA", True)
        return TrainingProfile(
            training_mode="QLoRA 4bit NF4 CUDA" if use_qlora
                          else "FP16 LoRA CUDA",
            learning_rate=2e-4 if use_qlora else 1e-4,
        )
    if device_type == "mps":
        # Apple Silicon: 속도보다 학습 안정성 우선 -> FP32 LoRA
        return TrainingProfile(training_mode="FP32 LoRA Apple MPS")
    # CPU: FP32 LoRA, max_length 256으로 축소
    return TrainingProfile(training_mode="FP32 LoRA CPU")
```

실습 코드 - LoRA Config & SFTTrainer 설정

PEFT 기초 실습 · 3/5 · 실습 코드 (train_lora.py #2 · Config)

```
def create_lora_config(profile):
    return LoraConfig(
        r=profile.lora_rank,              # 기본 16
        lora_alpha=profile.lora_alpha,    # 기본 32
        lora_dropout=profile.lora_dropout,
        bias="none",
        task_type="CAUSAL_LM",
        target_modules=[
            "q_proj", "k_proj", "v_proj", "o_proj",
            "gate_proj", "up_proj", "down_proj",
        ],
    )

def create_sft_config(profile, output_dir):
    return SFTConfig(
        output_dir=str(output_dir),
        num_train_epochs=profile.epochs,
        learning_rate=profile.learning_rate,
        eval_strategy="epoch", save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="eval_loss",
        # conversational 데이터에서 assistant 응답만 loss 계산
        assistant_only_loss=True,
        packing=False,
    )
```

실습 코드 - 학습 실행 및 저장

PEFT 기초 실습 · 3/5 · 실습 코드 (train_lora.py #3 · 학습 실행)

```
dataset = load_training_dataset(train_path, validation_path)
tokenizer = load_tokenizer()
model = load_model(profile)
lora_config = create_lora_config(profile)

trainer = SFTTrainer(
    model=model,
    args=create_sft_config(profile, output_dir),
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    processing_class=tokenizer,
    peft_config=lora_config,
)

train_result = trainer.train()

# load_best_model_at_end에 의해 선택된
# Validation 기준 최적 Adapter를 저장
trainer.save_model(str(output_dir))
tokenizer.save_pretrained(str(output_dir))
# -> models/hr-qwen-lora/ 에 저장됨
```

실습 코드 - 평가 스크립트 구조 (Base vs LoRA)

PEFT 기초 실습 · 3/5 · 실습 코드 (evaluate_model.py #1 · 비교 구조)

```
# scripts/evaluate_model.py
base_model = AutoModelForCausalLM.from_pretrained(MODEL_NAME)
tuned_model = PeftModel.from_pretrained(
    base_model, str(ADAPTER_PATH), is_trainable=False,
)

for record in evaluation_records:
    question = record["question"]

    # Base 모델: 같은 PeftModel에서 Adapter를 잠시 끈다
    with tuned_model.disable_adapter():
        base_answer = generate_answer(
            tuned_model, tokenizer, question,
        )

    # Fine-tuned: Adapter를 다시 켠 동일 모델
    tuned_answer = generate_answer(
        tuned_model, tokenizer, question,
    )

    base_result = evaluate_answer(base_answer, record)
    tuned_result = evaluate_answer(tuned_answer, record)
```

실습 코드 - 평가 지표 계산

PEFT 기초 실습 · 3/5 · 실습 코드 (evaluate_model.py #2 · 지표)

```
# Keyword 반영률: must_include 중 실제 포함된 비율
def keyword_score(answer, must_include, must_not_include):
    included = [k for k in must_include if k in answer]
    score = len(included)/len(must_include) if must_include else None
    forbidden = [k for k in must_not_include if k in answer]
    if forbidden and score is not None:
        score = max(0.0, score - 0.5)
    return score

# Unknown Policy 질문에서 "모른다"고 안전하게 답했는지 확인
def hallucination_score(answer, category, must_not_include):
    if category != "unknown_policy":
        return {"applicable": False}
    safe = any(t in answer for t in UNKNOWN_SAFE_TERMS)
    risky = any(t in answer for t in UNKNOWN_RISKY_PATTERNS)
    score = 1.0 if safe and not risky else 0.5 if safe else 0.0
    return {"applicable": True, "score": score}

# 종합 점수  일반 질문: keyword*0.7 + F1*0.3
#           Unknown 질문: keyword*0.3 + F1*0.2 + safety*0.5
```

실습 코드 - 실행 순서 및 결과 확인

PEFT 기초 실습 · 3/5 · 실습 코드 (실행 커맨드 요약)

```
# 1) 데이터셋 생성 (Context-based QA SFT)
python scripts/build_training_dataset.py

# 2) 데이터 분리·검증 (Instruction SFT 형식 확인)
python scripts/build_sft_dataset.py

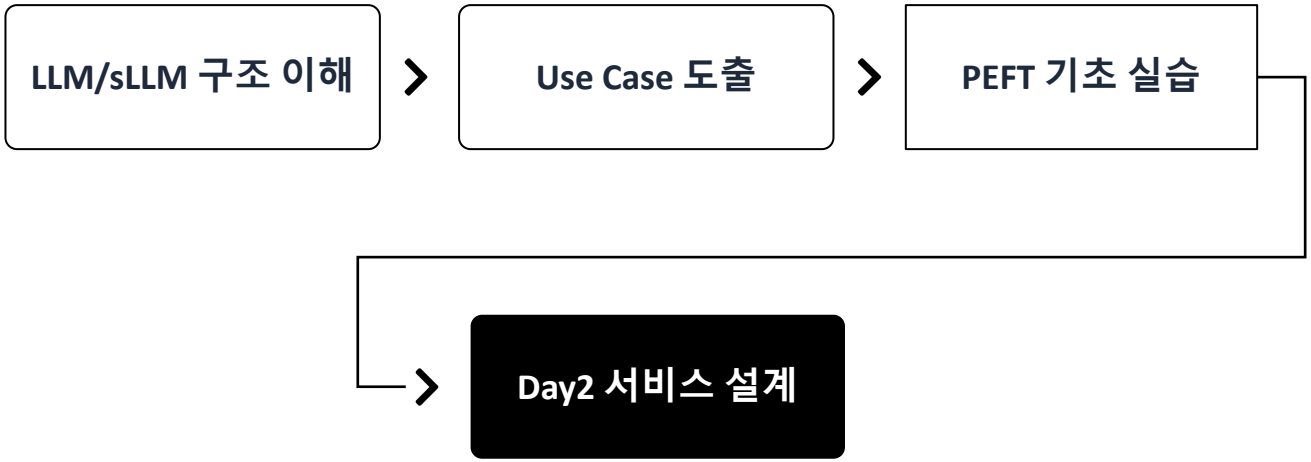
# 3) LoRA/QLoRA 학습 실행
SFT_MAX_LENGTH=512 SFT_EPOCHS=3 \
SFT_LEARNING_RATE=1e-4 SFT_GRAD_ACCUM=8 \
python scripts/train_lora.py
# -> models/hr-qwen-lora/ 생성

# 4) 결과 평가
python scripts/evaluate_model.py
# -> outputs/evaluation_results.jsonl
# -> outputs/evaluation_summary.json

# 재학습 시
rm -rf models/hr-qwen-lora
```

Day 1 마무리 · Day 2 예고

🏁 **Day1에서 이해한 LLM/sLLM 구조와 PEFT 기초는 Day2 서비스 설계·배포 실습의 기반이 됩니다.**



2. sLLM 서비스 설계 및 PEFT 응용

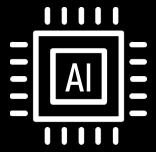
- sLLM 및 PEFT 트렌드
- PEFT 시나리오 도출
- sLLM 서비스 설계
- sLLM 서비스 구현
- PEFT 응용 실습

SK 주식회사 AX

sLLM 서비스 설계 및 PEFT 응용

RAG와 Fine-Tuning의 경계부터, 맥락 압축·파라미터 학습 코드 모듈, AIOps 연결까지 완성

교육 기간



2일 · 16시간 (Day1+Day2)

교육 방식



이론 50% · 실습 50%

Day2 목표



sLLM 설계 · PEFT 활용

도메인 지식 주입은 RAG(벡터DB 검색)와 Fine-Tuning(파라미터 학습)이라는 서로 다른 층위의 작업입니다. 이 둘의 구분이 Day2 전체의 출발점입니다.

Day1 → Day2 전환

Day 1 : What
sLLM이 왜 필요하고, 어떤 상황에서 선택되는지 이해

Day 2 : How
RAG와 Fine-Tuning의 경계, 목적별 기법 선택, 맥락 압축·파라미터 학습 실전 코드

Day2 핵심 산출물

목적에 맞는 파이프라인/임베딩 선정 + sLLM 서비스 구현 + PEFT 적용 및 서빙 배포 완료

Day 2 목차 한눈에 보기

목차	주요 학습내용 (이론 + 실전 보강)	시간	방식
sLLM 및 PEFT 트렌드	성능·효율 향상 3대 축(파라미터/Context&Reasoning/추론효율화) 연구동향 + 핵심 통찰	1h	이론
sLLM 목적별 PEFT기법 선택 가이드	RAG/벡터DB vs Fine-Tuning 비교 + 목적별 기법·파이프라인·임베딩 선택	1h	이론
PEFT 시나리오 도출	맥락압축+파인튜닝 결합 시나리오 + 도출 프레임워크 + 실전 예시	1h	이론
sLLM 서비스 설계	파이프라인 아키텍처 + 한국어 특화 대안모델(EXAONE 3.5) + 벡터DB 선정 기준	1h	이론
sLLM 서비스 구현	Fine-Tuning 필요 카테고리 + 맥락압축/파라미터학습 코드 모듈 + SOTA 패턴표	1h	이론/실습
PEFT 응용 실습	실습 시나리오 + 파이프라인 조립 코드 + 배포 체크리스트	3h	종합실습

sLLM 및 PEFT 트렌드

sLLM 및 PEFT 트렌드 · 1/4

- sLLM 및 PEFT 관련 학계/업계/현장 최신 동향
- Day1이 '왜 sLLM을 선택하는가(What)'였다면, Day2는 '현장에서 어떻게 구현하는가(How)'를 다룹니다.
- 파라미터 학습 기법과 맥락 압축 기법이 각각 어떻게 발전해왔는지는 다음 슬라이드에서 다룹니다.
- 다음 슬라이드에서 기법 진화 흐름, 핵심 통찰, Day2 실전 로드맵을 다룹니다.

연구 동향 - 성능·효율 향상의 3가지 축

sLLM 및 PEFT 트렌드 · 2/4 · 동향

sLLM의 성능과 효율은 서로 다른 3가지 축에서 동시에 발전해왔습니다.

축 1 · 모델 성능 향상

(파라미터 튜닝/학습)

- Adapter (2019) - 레이어 사이에 소형 모듈 삽입
- LoRA (2021) - 저랭크 행렬로 가중치 변화량만 학습
- QLoRA (2023) - 4비트 양자화 + LoRA 결합
- DoRA (2024) - 크기·방향 분해로 표현력 개선

축 2 · Context & Reasoning

(추론 및 컨텍스트 처리)

- Retrieval 기반 압축 - 관련 문서만 선별해 길이 축소
- Prompt Compression (LLMLingua 계열)
- 요약 기반 컨텍스트 압축
- Long-Context 모델 자체 확장 (예: 128K 토큰급)

축 3 · 추론 효율화

(Inference Optimization)

- Quantization - 가중치 정밀도 축소(INT8/INT4)
- MoE - 필요한 Expert만 선택적으로 연산
- Speculative Decoding - Draft 모델로 생성 가속
- KV Cache 최적화 - 반복 연산 재사용·경량화

핵심 통찰 - 기법 선택이 성능을 좌우한다.

sLLM 및 PEFT 트렌드 · 3/4 · 핵심 통찰

- 동일한 Open Weight sLLM을 선택하고 동일하게 추가 학습·Fine-Tuning을 적용하더라도, 결과 성능은 팀마다 크게 달라집니다
- 차이를 만드는 것은 모델 자체가 아니라, (1) 어떤 파라미터를 얼마나 학습시키는지, (2) 맥락을 어떻게 이해시키는지(Context & Reasoning), (3) 얼마나 효율적으로 추론하는지(Inference Optimization)에 대한 '방법론'입니다
- 즉, sLLM 구축의 성패는 모델 선택보다 학습·압축 기법의 설계 역량에 달려 있습니다

이 통찰이 Day2 전체를 관통하는 전제입니다 - 이후 모든 슬라이드는 '무엇을 쓸까'가 아니라 '어떻게 다룰까'에 초점을 둡니다.

Day2 실전 흐름 미리보기



sLLM 및 PEFT 트렌드 · 4/4 · 로드맵

기법 선택가이드 › 시나리오 도출 › 서비스 설계 › 서비스 구현 › **PEFT 응용실습**

- RAG와 Fine-Tuning의 경계를 먼저 명확히 하고(선택가이드), 이를 결합한 상용 수준 시나리오를 설계합니다.(시나리오 도출)
- Qwen2.5-1.5B + BGE-M3를 기준으로 서비스 아키텍처를 설계·구현하며, 이 과정에서 필요한 맥락압축/파라미터 학습 코드 모듈을 직접 다룹니다.
- 마지막 PEFT 응용실습에서 전체 파이프라인을 조립하고 배포까지 진행합니다.

sLLM 목적별 PEFT기법 선택 가이드

목적별 PEFT 기법 선택 · 1/5

- 목적별(도메인 특화 지식 주입, 극소 파라미터 사용, 추론 능력 강화) PEFT 기법 선택 기준
- 도메인 지식 주입은 RAG/벡터DB 파이프라인과는 명확히 구분되는 작업입니다 - 다음 슬라이드에서 비교표로 정리합니다
- 동일 목적(내부보안/도메인버티컬)이라도 선택한 모델 특성에 따라 필요한 후처리 수준이 달라짐을 함께 다룹니다
- 다음 슬라이드에서 RAG vs Fine-Tuning 비교, 목적별 기법·파이프라인·임베딩 선택, 시행착오 노하우를 다룹니다

도메인 지식 주입의 두 갈래 - RAG/벡터DB vs Fine Tuning

목적별 PEFT 기법 선택 · 2/5 · 핵심 구분

구분	RAG / 벡터DB 파이프라인	Fine-Tuning (파라미터 학습)
지식 저장 위치	외부 벡터DB (런타임에 검색)	모델 파라미터 내부 (학습 시 내재화)
지식 최신성 반영	문서만 갱신하면 즉시 반영	재학습해야 반영 (비용·시간 소요)
작동 방식	검색된 근거 문서를 컨텍스트로 제공	모델 자체의 이해·응답 능력이 향상
목적	사실 검색, 근거 제시, 최신 정보 반영	정확성·성능 자체의 근본적 향상
한계	검색 품질에 성능이 좌우됨	재학습 비용, 과적합 위험
관계	상호 배타적이지 않음	RAG + Fine-Tuned 모델 결합 가능

목적별 파이프라인·임베딩·기법 선택 가이드



목적별 PEFT 기법 선택 · 3/5 · 실전 가이드

목적	파이프라인 특성	임베딩 선택	적합 PEFT 기법
내부 보안 (문서 유출 방지)	완전 내부 RAG 파이프라인	다국어 임베딩 (BGE-M3)	**Prompt/Prefix Tuning**
도메인 버티컬 (전문지식 심화)	Fine-Tuning 중심 파이프라인	도메인 특화 임베딩	**LoRA / QLoRA**
두 목적 결합 (내부+전문화)	RAG + Fine-Tuned 모델 결합	다국어+도메인 하이브리드	**LoRA + Prompt Tuning 병행**

목적별 sLLM·임베딩 선택 다양성

목적별 PEFT 기법 선택 · 4/5 · 모델 선택지

목적/환경	sLLM 선택 예시	임베딩 선택 예시
다국어 범용 (한국어 포함)	Qwen2.5-1.5B-Instruct (3B~7B 확장 가능)	BGE-M3 (Dense·Sparse·Multi-Vector)
한국어 특화 강화	EXAONE 3.5 2.4B (LG AI Research, 한/영 균형 vocab)	BGE-M3 또는 한국어 특화 임베딩
On-Device 극경량	Qwen2.5-0.5B/1.5B급 경량 모델	경량 Dense 임베딩 위주

이번 과정은 Qwen2.5-1.5B + BGE-M3를 기준으로 진행합니다. 학습 파라미터를 3B~7B로 올려도 동일한 코드 구조를 그대로 활용할 수 있도록 설계된 조합입니다.

실전 시행착오별 노하우

목적별 PEFT 기법 선택 · 5/5 · 노하우

- RAG만으로 해결하려다 실패하는 경우 - 근거 문서가 없는 질문(추론·판단이 필요한 질문)에는 RAG가 근본적으로 취약함
- Fine Tuning만으로 해결하려다 실패하는 경우 - 최신 정보나 자주 바뀌는 규정은 재학습 주기가 못 따라감
- 임베딩 모델과 sLLM의 언어 커버리지 불일치 - 임베딩은 다국어인데 sLLM이 한국어에 약하면 검색은 잘 되어도 답변 품질이 떨어짐
- 처음부터 완벽한 파이프라인을 설계하려 하지 말고, 작은 시나리오로 시작해 RAG/Fine-Tuning 비중을 점진적으로 조정하는 것이 현장에서 검증된 접근

PEFT 시나리오 도출

PEFT 시나리오 도출 · 1/4

- sLLM 목적별 PEFT 시나리오 도출 및 시나리오 간 차이점 이해
- 맥락 압축과 파라미터 Fine-Tuning을 결합하면 기존과는 다른 레벨의 상용 sLLM을 구축할 수 있습니다
- 이 결합 구조와 시나리오 도출 프레임워크는 다음 슬라이드에서 다룹니다
- 다음 슬라이드에서 결합 아키텍처, 도출 프레임워크, 실전 예시를 다룹니다

새로운 레벨의 sLLM 상용 구축 - 맥락압축 + 파인튜닝 결합

PEFT 시나리오 도출 · 2/4 · 핵심 개념

기존 방식 (RAG 기반 확장)

RAG(벡터DB 검색)만 적용 - 근거 문서를 그대로 컨텍스트에 삽입, 모델 자체는 변경 없음

새로운 레벨 (LLM 연구 확장)

맥락 정보를 압축해 효율적으로 제시 + 파라미터 Fine Tuning으로 모델 자체의 이해도를 함께 끌어올림

동일한 Open Weight sLLM이라도, 맥락을 어떻게 압축해 제시하고 파라미터를 어떻게 학습시키느냐에 따라 결과 수준이 완전히 달라집니다. 이것이 '상용 수준'과 '데모 수준'을 가르는 지점입니다.

시나리오 도출 프레임워크

PEFT 시나리오 도출 · 3/4 · 프레임워크

1. **목적 정의**
   내부 보안 / 도메인 버티컬 / 두 목적 결합 중 무엇을 우선할지 명확히 함
2. **데이터 확보**
   도메인 문서(RAG용)와 학습 데이터(Fine-Tuning용)를 구분해 수집
3. **기법 선택 (압축·파인튜닝 비중 결정)**
   맥락압축 위주로 갈지, 파라미터 학습 비중을 높일지 목적에 맞게 배분
4. **검증 기준 설정**
   정확도·응답속도·비용 등 무엇을 기준으로 성공을 판단할지 사전 정의

실전 시나리오 예시 - Qwen2.5-1.5B 기준 도메인 버티컬 구축

PEFT 시나리오 도출 · 4/4 · 실전 예시

목적	사내 계약서 검토 보조 sLLM 구축 (법무 도메인 버티컬)
데이터	계약서 조항 요약 데이터 + 계약 유형별 위험조항 판례 요약
기법 배분	맥락압축(긴 계약서 요약 후 제시) 60% + LoRA Fine-Tuning(법무 용어·판단기준 주입) 40%
검증 기준	위험조항 탐지 정확도, 응답 근거 제시 여부, 평균 응답 지연시간

sLLM 서비스 설계

sLLM 서비스 설계 · 1/4

- sLLM 서비스 파이프라인 및 임베딩 알고리즘 연결 구조, 벡터DB 선정 및 설계
- 실습 기준 모델: Qwen2.5-1.5B-Instruct + BGE-M3 임베딩 - 아키텍처는 다음 슬라이드에서 상세히 다룹니다.
- 한국어 환경에 특화된 대안 모델(EXAONE 3.5)도 함께 소개합니다.
- 다음 슬라이드에서 아키텍처 상세, 한국어 특화 모델 비교, 벡터DB 선정 기준을 다룹니다.

서비스 파이프라인 아키텍처 - Qwen2.5-1.5B + BGE-M3

sLLM 서비스 설계 · 2/4 · 아키텍처



- 질의를 BGE-M3로 임베딩 후 벡터DB에서 관련 문서를 검색(Dense+Sparse 하이브리드 가능)
- 검색된 문서는 그대로 넣지 않고 맥락 압축을 거쳐 핵심만 추려 Qwen2.5-1.5B에 전달
- Qwen2.5-1.5B는 LoRA로 도메인 지식이 주입된 상태로 최종 응답을 생성
- 이 구조는 RAG(검색)와 Fine-Tuning(파라미터 학습)이 함께 작동하는 결합형 파이프라인입니다

한국어 특화 대안 모델 - EXAONE 3.5

sLLM 서비스 설계 · 3/4 · 모델 비교

비교 항목	Qwen2.5-1.5B-Instruct	EXAONE 3.5 (2.4B~)
개발사	Alibaba (Qwen Team)	LG AI Research
파라미터 규모	1.5B (3B~7B 확장 가능)	2.4B / 7.8B / 32B
언어 구성	29개 이상 언어 (다국어 범용)	한국어·영어 약 50:50 균형 Vocabulary
강점	폭넓은 다국어 지원, 활발한 생태계	한국어 실사용 벤치마크에서 동급 대비 우수
컨텍스트 길이	최대 128K (통상 32K)	최대 32K

한국어 응답 비중이 특히 높은 서비스라면 EXAONE 3.5를, 다국어 확장성과 생태계 크기를 우선한다면 Qwen2.5 계열을 검토합니다.

벡터DB 선정 기준

sLLM 서비스 설계 · 4/4 · 선정 기준

- 완전 내부 운용 필요 여부 - 사내 인프라에 직접 설치 가능한 오픈소스 벡터DB인지 확인
- 하이브리드 검색 지원 여부 - BGE-M3의 Dense+Sparse 결과를 함께 활용할 수 있는지
- 메타데이터 필터링 성능 - 부서/문서유형 등 조건별 검색이 얼마나 유연한지
- 운영 복잡도 - 소규모 PoC 단계에서는 가벼운 임베디드형, 상용 확장 단계에서는 분산 클러스터형 검토
- [여기에 팀에서 검토 중인 벡터DB 후보와 비교 결과를 채워 넣으세요]

sLLM 서비스 구현

sLLM 서비스 구현 · 1/5

- sLLM 서비스 파이프라인 구현, 벡터DB 연동, 기본 성능 튜닝
- 상용 LLM은 벤더가 이미 상당한 후처리(정확성·안전성 강화)를 적용해 제공하지만, sLLM은 이를 사용자가 직접 구현해야 합니다
- 이를 위해 필요한 Fine-Tuning 카테고리와 실무에서 자주 쓰이는 코드 모듈은 다음 슬라이드에서 다룹니다
- 다음 슬라이드에서 필요 카테고리, 맥락압축 코드, 파라미터학습 코드, SOTA 패턴표를 다룹니다

Fine-Tuning이 필요한 카테고리 정리

sLLM 서비스 구현 · 2/5 · 개념

카테고리	상용 LLM은 이미 처리한 부분	sLLM에서 직접 다뤄야 할 기법
정확성 향상	도메인 용어·사실 관계 오류 감소	**LoRA / QLoRA (파라미터 학습)**
맥락 활용 효율	긴 문서를 이해하고 핵심만 반영	**Prompt Compression / 요약 압축**
응답 스타일 정렬	사내 톤·형식에 맞는 응답 생성	**Instruction Tuning**
안전성/정책 준수	금지 주제 회피, 민감정보 마스킹	**규칙 기반 후처리 + 소규모 Fine-Tuning**

맥락 정보 압축 기법 코드 모듈

sLLM 서비스 구현 · 3/5 · 코드 모듈 (개념 예제) - Context Compression #1

```
# pip install -U torch transformers accelerate pypdf python-docx
import os
import torch
from pypdf import PdfReader
from docx import Document
from transformers import AutoTokenizer
from transformers import AutoModelForCausalLM

# 1. Qwen2.5-1.5B 로드
MODEL_NAME = "Qwen/Qwen2.5-1.5B"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# GPU(CUDA) 환경
# model = AutoModelForCausalLM.from_pretrained(
#     MODEL_NAME,
#     torch_dtype=torch.float16,
#     device_map="auto",
# )

# CPU 환경 (Mac / Windows 공통)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    torch_dtype=torch.float32,
)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.pad_token_id
```

맥락 정보 압축 기법 코드 모듈

sLLM 서비스 구현 · 3/5 · 코드 모듈 (개념 예제) - Context Compression #2

```
# 2. 공통 생성 함수
def generate_text(prompt, max_new_tokens=512):
    inputs = tokenizer(
        prompt, return_tensors="pt",
        truncation=True, max_length=2048,
    )
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False, use_cache=True,
            pad_token_id=tokenizer.pad_token_id,
            eos_token_id=tokenizer.eos_token_id,
        )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

# 실제 compress_context() · RAG 검색 · FastAPI 엔드포인트까지
# 이어지는 전체 구현은 "PEFT 응용 실습"에서 sllm_service
# 패키지의 실제 코드로 다룹니다.
```

Context & Reasoning 추가 예제 - Long-Context 처리 (YaRN)

sLLM 서비스 구현 · 코드 모듈 보강 (개념 예제) - Long Context 처리

```
# pip install -U torch transformers accelerate
from transformers import AutoConfig, AutoTokenizer, AutoModelForCausalLM

# Long Context 확장 설정 (YaRN)
MODEL_NAME = "Qwen/Qwen2.5-1.5B-Instruct"
config = AutoConfig.from_pretrained(MODEL_NAME)

# 기본 32K context를 약 4배 확장하는 예시
# 실제 안정성은 모델/transformers 버전에 따라 달라질 수 있음
config.rope_scaling = {
    "rope_type": "yarn",
    "factor": 4.0,
    "original_max_position_embeddings": 32768,
}

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME, config=config, torch_dtype=torch.float32,
)

# 검색 없이 문서 전체를 그대로 입력에 넣는 Long-Context 방식으로,
# RAG/Context Compression과는 다른 트레이드오프를 가집니다.
```

파라미터 학습 기법(QLoRA) 코드 모듈

sLLM 서비스 구현 · 4/5 · 코드 모듈 (QLoRA - Day1 실습 참고)

```
# QLoRA 전체 학습 코드는 이미 Day1 실습
# scripts/train_lora.py 에서 실제로 다뤘습니다.
# CUDA 환경에서 USE_QLORA=True(기본값)이면 자동으로
# 아래 4bit NF4 양자화 로딩으로 분기합니다.

from transformers import BitsAndBytesConfig
from peft import prepare_model_for_kbit_training

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    quantization_config=quantization_config,
    device_map="auto",
)
model = prepare_model_for_kbit_training(model)

# 이후 LoRA Config·SFTTrainer 적용은 train_lora.py #2,#3과 동일
```

SOTA 패턴 & 연구 동향 정리표

sLLM 서비스 구현 · 5/5 · 정리표

기법	핵심 아이디어	현장 적용 상황
QLoRA	4비트 양자화 + LoRA	메모리 제약 환경에서 대형 모델 파인튜닝
DoRA	가중치 크기·방향 분리 학습	LoRA보다 높은 표현력이 필요한 경우
Prompt Compression	중요도 기반 토큰 제거	긴 문서를 짧은 컨텍스트로 압축
Instruction Tuning	지시-응답 쌍 학습	응답 스타일·형식을 사내 기준에 정렬

추론 효율화 (Inference Optimization)

추론 효율화 · 1/6 · 세 번째 축

- LLM 자체의 연산·구조를 최적화해 추론 속도와 서빙 비용을 개선하는 작업
- 파라미터 튜닝(모델 성능 향상)이나 Context & Reasoning(맥락 처리)과는 다른 층위 - '어떻게 더 빠르고 싸게 실행하는가'의 문제
- 대표 기법: Quantization, MoE(Mixture of Experts), Speculative Decoding, KV Cache 최적화
- 다음 슬라이드에서 3가지 축 전체 프레임워크와 4가지 핵심 기법을 각각 다룹니다.

성능·효율 최적화의 3가지 축 - 전체 프레임워크

추론 효율화 · 2/6 · 프레임워크

기준 축	핵심 질문	대표 기법
① **모델 성능 향상 (파라미터 튜닝/학습)**	정확성을 어떻게 높이는가?	LoRA, QLoRA, DoRA
② **Context & Reasoning (추론 및 컨텍스트 처리)**	맥락을 어떻게 이해시키는가?	Prompt Compression, RAG, Long-Context
③ **추론 효율화 (Inference Optimization)**	어떻게 더 성능을 높이고 자원 비용 절감을 할 수 있는가?	**Quantization, MoE, Speculative Decoding, KV Cache**

세 축은 서로 배타적이지 않고 함께 적용됨 - 실전에서 LoRA(①) + 맥락압축(②) + Quantization(③)을 동시에 조합하는 경우가 많습니다.

Quantization - 모델 경량화

추론 효율화 · 3/6 · 기법 1

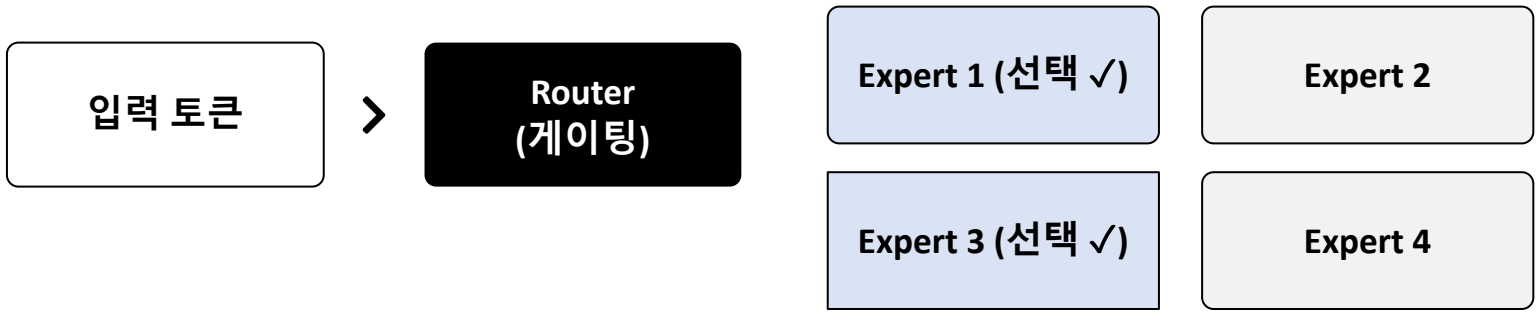
- 가중치·활성값의 정밀도를 FP16/FP32에서 INT8/INT4 등으로 낮춰 메모리 사용량과 연산량을 줄이는 기법
- PTQ(Post-Training Quantization) - 학습 후 양자화, 적용이 간단하지만 정확도 손실 가능
- QAT(Quantization-Aware Training) - 학습 과정에 양자화를 반영해 정확도 손실을 최소화

정밀도	메모리 사용량	추론 속도	정확도 영향
FP16	기준(100%)	기준(1x)	정확도 손실 없음
INT8	약 50%	약 1.5~2x	경미한 손실
INT4 (예: GPTQ, AWQ)	약 25%	약 2~3x	미세 조정 없이는 손실 발생 가능

MoE (Mixture of Experts) - 선택적 연산 구조

추론 효율화 · 4/6 · 기법 2

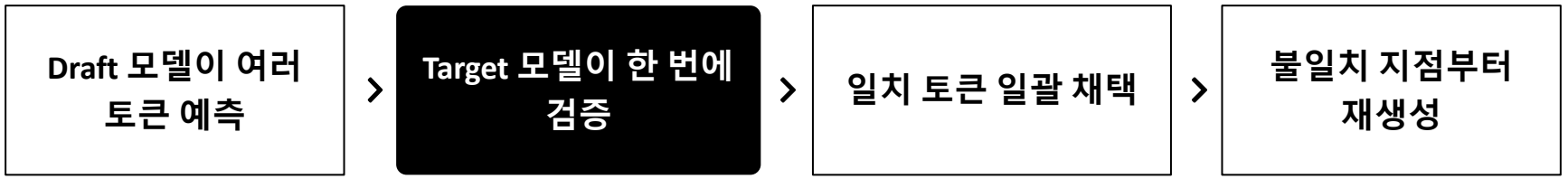
입력마다 전체 파라미터가 아닌 일부 Expert만 활성화



- Router가 입력 특성에 맞는 소수의 Expert만 선택해 연산 - 전체 파라미터는 크지만 실제 활성화되는 파라미터는 일부
- 대형 모델 수준의 표현력을 유지하면서도 추론 시 연산량은 훨씬 적게 유지 가능
- 최근 open-weight LLM/sLLM 계열에서 MoE 구조 채택이 늘어나는 추세

Speculative Decoding - 추론 속도 가속

추론 효율화 · 5/6 · 기법 3



- 작고 빠른 Draft 모델이 다음 여러 토큰을 미리 순차 예측 (저비용 추론)
- 크고 정확한 Target 모델(예: Qwen2.5-1.5B)이 이 후보들을 한 번의 순전파로 병렬 검증
- 검증을 통과한 토큰은 그대로 채택해 여러 토큰을 한 번에 생성한 효과를 얻음 - 최종 출력 품질은 Target 모델 기준으로 유지
- 불일치가 발생한 지점부터는 Target 모델이 다시 생성하여 정확성을 보장

KV Cache 최적화 - 메모리 효율적 추론

추론 효율화 · 6/6 · 기법 4

- Transformer는 매 토큰 생성 시 이전 토큰들의 Key/Value를 다시 계산하지 않도록 캐시(KV Cache)에 저장해 재사용
- 하지만 시퀀스가 길어질수록 캐시 크기가 선형으로 증가해 메모리 병목이 발생
- 이를 줄이기 위한 최적화 기법들이 활발히 연구되고 있습니다

기법	핵심 아이디어	적합한 상황
PagedAttention	캐시를 페이지 단위 관리 메모리 단편화 최소	긴 대화/다중 요청 동시 처리 서빙
Multi-Query Attention (MQA)	여러 Query가 Key/Value 공유, 캐시 크기 축소	메모리 제약이 큰 On-Device 환경
Grouped-Query Attention (GQA)	MQA와 기존 방식의 절충 - 그룹 단위 KV 공유	Qwen2.5 등 최근 sLLM에서 채택

PEFT 응용 실습

PEFT 응용 실습 · 1/4

STEP 1

서비스 구현 및 PEFT 적용

- Qwen2.5-1.5B-Instruct + BGE-M3 기반 서비스 파이프라인 구현
- 목적에 맞는 PEFT 기법(LoRA 등) 적용
- 적용 전/후 응답 품질 비교

STEP 2

sLLM 서빙 배포

- 튜닝된 모델을 서빙 환경에 배포
- 기본 성능(지연시간/처리량) 점검
- 실서비스 관점의 체크리스트 점검

실습 시나리오 최종 정의

PEFT 응용 실습 · 2/4 · 실습 시나리오

목표

Qwen2.5-1.5B-Instruct + BGE-M3 + hr-qwen-lora로 HR 문서 기반 Q&A 서비스 완성

기법 구성

sllm_service 패키지: RAG 검색 + Context Compression + LoRA 자동 로딩 통합

배포 목표

FastAPI로 서빙하며, Day1에서 학습한 hr-qwen-lora가 자동으로 연결되는지 확인

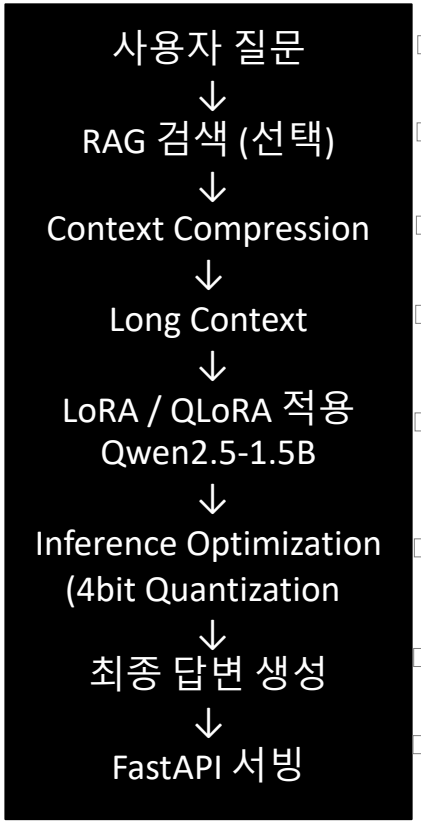
실습 진행 순서

1. **Day1 결과물 연결**
   scripts로 만든 hr-qwen-lora Adapter를 sllm_service가 자동 감지
2. **파이프라인 조립**
   config.py → runtime.py → routers/services 흐름을 실제 코드로 확인 (다음 슬라이드)
3. **실행 및 테스트**
   uvicorn으로 서비스 실행 후 Swagger에서 통합 질의 테스트

실습 코드 - 파이프라인 조립

PEFT 응용 실습 · 3/4 · 데모 코드 (Skeleton 설계)

RAG는 문서를 찾고, Context Compression은 찾은 문서를 줄이고, LoRA/QLoRA 모델은 도메인에 맞는 방식으로 답변



코드 단계	관련 기술	설명
bge_m3.encode()	Retrieval Embedding	질문을 벡터로 변환
vector_db.search()	RAG / Vector Search	관련 문서 검색 (선택 기능)
compress_context()	Context Compression	검색 결과 또는 원문을 LLM 처리 쉬운 맥락으로 압축
Long Context	Long Context	긴 문서 압축 없이 그대로 입력
LoRA / QLoRA	Parameter Tuning	도메인 특화 성능 향상
4bit Quantization → GPU 필요	Inference Optimization	GPU 메모리 절감 및 추론 효율 향상
qwen_model.generate()	LLM Generation	최종 자연어 생성
FastAPI Endpoint	Serving System	외부 서비스 제공

실습 코드 - 파이프라인 조립

PEFT 응용 실습 · 3/4 · 데모 코드 (Skeleton #1)

```
# LLM Application Pipeline Skeleton
def answer_query(
    user_query: str,
    mode: str = "compression",     # long_context | compression
    use_rag: bool = False,
) -> str:
    # 1. Retrieval Embedding (선택)
    retrieved_docs = None
    if use_rag:
        q_emb = bge_m3.encode(
            user_query, normalize_embeddings=True,
        )
    # 2. Vector Search (선택)
        retrieved_docs = vector_db.search(
            q_emb,
            top_k=5,
        )
    # 3. Context Engineering
    if mode == "compression":
        context = compress_context(
            docs=retrieved_docs, model=qwen_model, max_tokens=500,
        )
    elif mode == "long_context":
        context = build_long_context(
            docs=retrieved_docs,
        )
    else:
        raise ValueError("Unknown Context Mode")
```

sllm_service 패키지 내부 구조

PEFT 응용 실습 · 3/4 · 데모 코드 (sllm_service 구조)

```
sllm_service/
├── app.py              FastAPI 생성 + Router 조립
├── config.py           모델명·경로·환경변수 설정
├── runtime.py          모델·임베딩 로딩, LoRA 자동 연결
├── state.py            문서/벡터 인덱스 메모리 상태
├── schemas.py          요청 데이터 형식(Pydantic)
│
├── services/           비즈니스 로직
│   ├── generation_service.py   Prompt 구성·생성
│   ├── document_service.py     문서 Chunk·Embedding
│   ├── rag_service.py          Top-K 검색
│   └── vector_service.py       벡터 인덱스 추상화
│
└── routers/            HTTP API 경로
    ├── system_router.py      Health Check
    ├── document_router.py    업로드·Chunk 조회
    ├── rag_router.py         RAG·압축·Long Context
    └── inference_router.py   추론 최적화·통합 Pipeline
```

실습 코드 - app.py (FastAPI 앱 조립)

PEFT 응용 실습 · 3/4 · 데모 코드 (app.py)

```
# sllm_service/app.py
from fastapi import FastAPI
from sllm_service.config import settings
from sllm_service.routers.document_router import router as document_router
from sllm_service.routers.inference_router import router as inference_router
from sllm_service.routers.rag_router import router as rag_router
from sllm_service.routers.system_router import router as system_router

def create_app() -> FastAPI:
    application = FastAPI(
        title=settings.app_title,
        version=settings.app_version,
    )
    application.include_router(system_router)
    application.include_router(document_router)
    application.include_router(rag_router)
    application.include_router(inference_router)
    return application

app = create_app()

# 실행: python -m uvicorn sllm_service.app:app --port 8080
```

실습 코드 - config.py #1 (기본 설정)

PEFT 응용 실습 · 3/4 · 데모 코드 (config.py #1)

```
# sllm_service/config.py
@dataclass(frozen=True)
class Settings:
    app_title: str = os.getenv("APP_TITLE", "sLLM-service")
    app_version: str = os.getenv("APP_VERSION", "1.3.0")

    model_name: str = os.getenv(
        "MODEL_NAME", "Qwen/Qwen2.5-1.5B-Instruct",
    )
    embedding_model_name: str = os.getenv(
        "EMBEDDING_MODEL_NAME", "BAAI/bge-m3",
    )

    max_input_tokens: int = parse_int_env("MAX_INPUT_TOKENS", 8192)
    chunk_size: int = parse_int_env("CHUNK_SIZE", 1000)

    # CUDA에서만 bitsandbytes 4bit NF4 적용, MPS/CPU는 자동 분기
    use_4bit: bool = parse_bool_env("USE_4BIT", default=True)

    # Qwen·BGE-M3를 모두 MPS에 올리면 메모리 부담이 커서
    # 임베딩 모델은 기본적으로 CPU 사용
    embedding_device: str = os.getenv("EMBEDDING_DEVICE", "cpu")
```

실습 코드 - config.py #2 (LoRA 선택 정책)

PEFT 응용 실습 · 3/4 · 데모 코드 (config.py #2)

```
# auto: Adapter 있으면 LoRA, 없으면 Base
# base: Adapter 있어도 Base 강제 사용  /  lora: Adapter 필수
model_variant: ModelVariant = parse_model_variant_env(
    "MODEL_VARIANT", default="auto",
)

# 기본 경로: models/hr-qwen-lora/  (Day1 학습 결과)
lora_adapter_path: Path = parse_path_env(
    "LORA_ADAPTER_PATH",
    default=PROJECT_ROOT / "models" / "hr-qwen-lora",
)

lora_adapter_name: str = os.getenv(
    "LORA_ADAPTER_NAME", "hr-assistant",
)

# False 권장: Base + PEFT Adapter 형태 유지 (병합 X)
merge_lora_adapter: bool = parse_bool_env(
    "MERGE_LORA_ADAPTER", default=False,
)

# True면 Adapter가 불완전할 때 서버 기동을 실패시킴
# 초급 실습에서는 False 권장 (자동으로 Base로 대체)
strict_lora_loading: bool = parse_bool_env(
    "STRICT_LORA_LOADING", default=False)
```

실습 코드 - runtime.py #1 (환경별 Base 모델 로딩)

PEFT 응용 실습 · 3/4 · 데모 코드 (runtime.py #1)

```
# sllm_service/runtime.py
def load_base_qwen_model(has_cuda, has_mps):
    if has_cuda and settings.use_4bit:
        quantization_config = BitsAndBytesConfig(
            load_in_4bit=True, bnb_4bit_quant_type="nf4",
            bnb_4bit_compute_dtype=torch.float16,
        )
        model = AutoModelForCausalLM.from_pretrained(
            settings.model_name,
            quantization_config=quantization_config, device_map="auto",
        )
        return model, model.device, "4bit NF4 CUDA"
    if has_cuda:
        model = AutoModelForCausalLM.from_pretrained(
            settings.model_name, dtype=torch.float16,
        ).to("cuda:0")
        return model, torch.device("cuda:0"), "FP16 CUDA"
    if has_mps:
        model = AutoModelForCausalLM.from_pretrained(
            settings.model_name, dtype=torch.float16,
        ).to("mps")
        return model, torch.device("mps"), "FP16 Apple MPS"
    # CUDA/MPS 모두 없으면 CPU FP32
    model = AutoModelForCausalLM.from_pretrained(
        settings.model_name, dtype=torch.float32,).to("cpu")
    return model, torch.device("cpu"), "FP32 CPU"
```

실습 코드 - runtime.py #2 (LoRA 자동 감지·연결)

PEFT 응용 실습 · 3/4 · 데모 코드 (runtime.py #2 · 핵심)

```
# 정상 Adapter 조건: adapter_config.json +
# adapter_model.safetensors(또는 .bin) 존재
def inspect_lora_adapter(adapter_path):
    if not adapter_path.is_dir():
        return False, "Adapter directory not found"
    config_ok = (adapter_path / "adapter_config.json").is_file()
    weight_ok = any(
        (adapter_path / name).is_file()
        for name in ("adapter_model.safetensors", "adapter_model.bin")
    )
    return (config_ok and weight_ok), None

def decide_lora_usage(adapter_path):
    available, _ = inspect_lora_adapter(adapter_path)
    if settings.model_variant == "base":
        return False, available
    if settings.model_variant == "lora":
        if not available:
            raise FileNotFoundError("정상 LoRA Adapter가 필요합니다.")
        return True, True
    # auto: Day1에서 hr-qwen-lora를 학습해 두면 자동으로 감지되어
    # Fine-tuned 모델이 응답합니다. 없으면 Base 모델로 대체됩니다.
    return available, available
```

실습 코드 - schemas.py & state.py

PEFT 응용 실습 · 3/4 · 데모 코드 (schemas.py · state.py)

```
# sllm_service/schemas.py
class QueryRequest(BaseModel):
    document_id: str
    question: str
    top_k: int = Field(5, ge=1, le=20)
    max_new_tokens: int = Field(256, ge=32, le=1024)

class PipelineRequest(BaseModel):
    document_id: str
    question: str
    mode: Literal["rag","compression","long_context"] = "compression"
    use_rag: bool = True
    top_k: int = Field(5, ge=1, le=20)
    max_new_tokens: int = Field(256, ge=32, le=1024)

# sllm_service/state.py
# 업로드한 문서와 벡터 인덱스를 저장하는 런타임 상태
documents: dict[str, dict] = {}
vector_indexes: dict[str, object] = {}
# Mac: NumPy 인덱스 / Colab·Linux: FAISS 인덱스 - 타입에 의존하지 않음
```

실습 코드 - services / routers 역할 정리

PEFT 응용 실습 · 3/4 · 데모 코드 (services · routers)

```
# services/  (비즈니스 로직)
generation_service.py   Prompt 구성 + Qwen 텍스트 생성
document_service.py     문서 읽기·Chunk 분할·Embedding 등록
rag_service.py           질문 Embedding + Top-K Chunk 검색
vector_service.py       Mac NumPy / Colab FAISS 검색 추상화

# routers/  (HTTP API)
system_router.py        루트 API + Health Check
document_router.py      샘플 로드·파일 업로드·Chunk 조회
rag_router.py            RAG · Compression · Long Context API
inference_router.py     추론 최적화 테스트 + 통합 Pipeline

# app.py는 이 4개 router를 include_router()로 조립만 하고,
# 실제 로직은 모두 services/ 아래에 있습니다.
```

실습 코드 - 실행 및 통합 테스트

PEFT 응용 실습 · 3/4 · 데모 코드 (실행 · 테스트)

```
# 1) 서비스 실행
python -m uvicorn sllm_service.app:app --host 127.0.0.1 --port 8080

# 2) Swagger UI 접속
http://127.0.0.1:8080/docs

# 3) 통합 Pipeline 테스트
POST /pipeline
{
  "document_id": "샘플 로드 후 받은 id",
  "question": "연차 신청은 어떻게 하나요?",
  "mode": "compression",
  "use_rag": true
}

# MODEL_VARIANT=auto 이면 models/hr-qwen-lora/가 있을 때
# Day1에서 학습한 Fine-tuned 모델이 자동으로 응답합니다.
```

배포 체크리스트

PEFT 응용 실습 · 4/4 · 체크리스트

- LoRA 적용 전/후 동일 질문 세트에 대한 응답을 비교해 개선을 확인했는가?
- 맥락 압축이 지나쳐 핵심 정보가 유실되지 않는지 샘플 검증을 거쳤는가?
- 평균 응답 지연시간이 서비스 요구 수준(예, 3초 이내)을 충족하는가?
- 예외 상황(검색 결과 없음, 토큰 초과 등)에 대한 처리 로직이 준비되었는가?

Day 2 마무리 - 과정의 철학

과정 철학

sLLM은 도메인 기반 버티컬 지식 주입에 특화된 AI 서비스입니다.

동일한 LLM 기술이라도 파라미터를 어떻게 학습시키고(모델 성능 향상), 맥락을 어떻게 이해시키며(Context & Reasoning), 얼마나 효율적으로 실행하느냐(추론 효율화)에 따라서 성능은 천차만별입니다.

이 세 축을 함께 설계하는 작업이 곧 사용자에게 특화된 정확성과 성능을 갖춘 핵심 sLLM으로 거듭나게 하는 과정입니다.

그래서 이 성능을 지속적으로 유지하려면

매번 사람이 직접 감시 › **Agentic AI가 자동 감지·대응** › **AIOps 체계로 연결**

사람이 매번 감시·재학습을 판단할 수는 없기에, 향후 Agentic AI가 성능 저하를 스스로 감지하고 대응하며 AIOps 체계로 자연스럽게 연결됩니다. AIOps는 별도 과목에서 학습합니다.